

Resolución del TP Grupal — ORM con JPA/Hibernate

Integrantes: Cortes Maria Pia, Fehler Valentina, Delahaye Augusto, Rodriguez Santos, Zerpa Santiago

1. Qué se resolvió

Se modeló el dominio de facturación con dos superclases mapeadas (`@MappedSuperclass`): **EntityId** (aporta el id con `@Id/@GeneratedValue` a todas las entidades) y **AuditoriaApp** (aporta fechaAlta, fechaBaja, fechaModificacion y los usuarios de auditoría). Sobre esa base se anotaron las 14 entidades del modelo con `@Entity`, y sus relaciones con `@ManyToOne`, `@OneToMany` y `@OneToOne` según correspondía.

El punto central de la consigna — persistir una FacturaVenta completa con un único `em.persist()` gracias al `cascade = CascadeType.ALL` configurado en `FacturaVenta.detalles` — se implementó y se verificó funcionando en el Main.

2. Conexión con PostgreSQL

La unidad de persistencia **FacturacionPU** (`persistence.xml`) quedó configurada así:

```
provider: org.hibernate.jpa.HibernatePersistenceProvider
driver: org.postgresql.Driver
url: jdbc:postgresql://localhost:5432/facturacion_db
usuario: postgres
hibernate.hbm2ddl.auto: update
```

Con `hibernate.hbm2ddl.auto = update`, al ejecutar el Main, Hibernate generó automáticamente todas las tablas a partir de las entidades anotadas — sin necesidad de escribir el DDL a mano.

3. Verificación en la base de datos

Se verificó en pgAdmin 4, dentro de la base `facturacion_db`, que la tabla **articulo** se creó con sus 11 columnas: `id`, `fechaalta`, `fechabaja`, `fechamodificacion`, `codigo`, `denominacion`, `usuariobaja`, `usuariocarga`, `usuariomodificado`, `marca_id` y `rubro_id`.

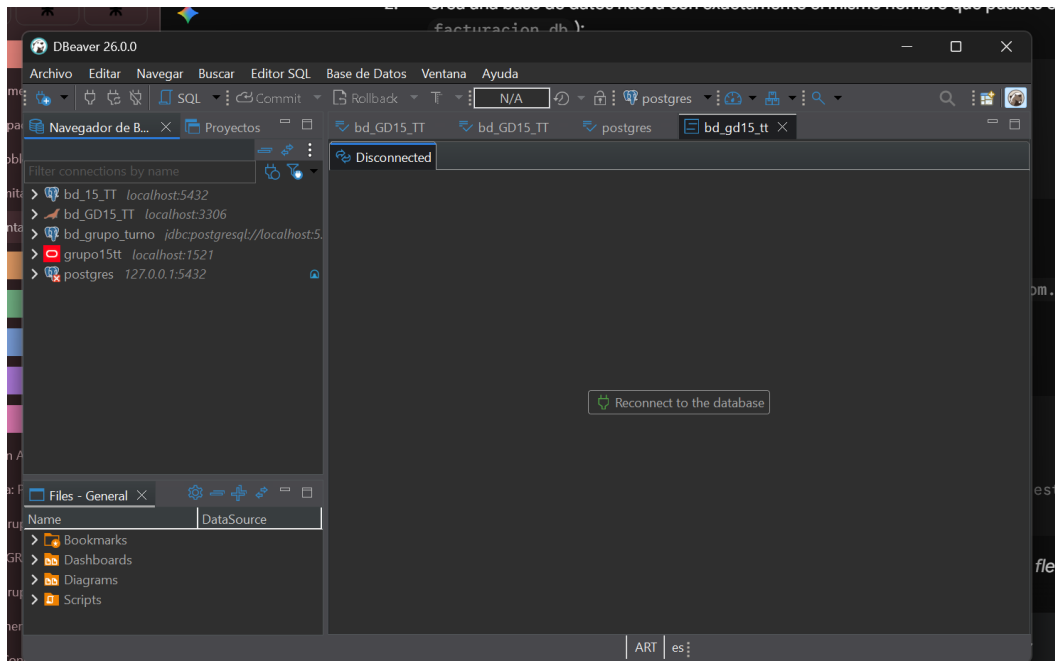
Ese número coincide exactamente con lo esperado: 1 columna de `EntityId` (`id`) + 6 columnas de `AuditoriaApp` (3 fechas + 3 usuarios) + 4 columnas propias de `Articulo` (`codigo`, `denominacion` y las claves foráneas `marca_id` / `rubro_id` generadas por sus relaciones `@ManyToOne`). Esto confirma que la herencia con `@MappedSuperclass` y las relaciones quedaron mapeadas correctamente.

4. Resultado

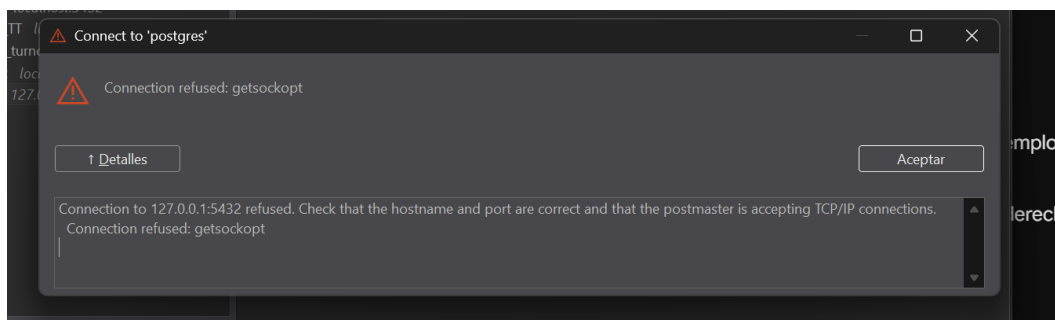
El TP quedó resuelto de punta a punta: entidades anotadas, `persistence.xml` apuntando a PostgreSQL, y la base de datos real reflejando el modelo tal como fue diseñado.

Anexo: proceso de conexión con PostgreSQL

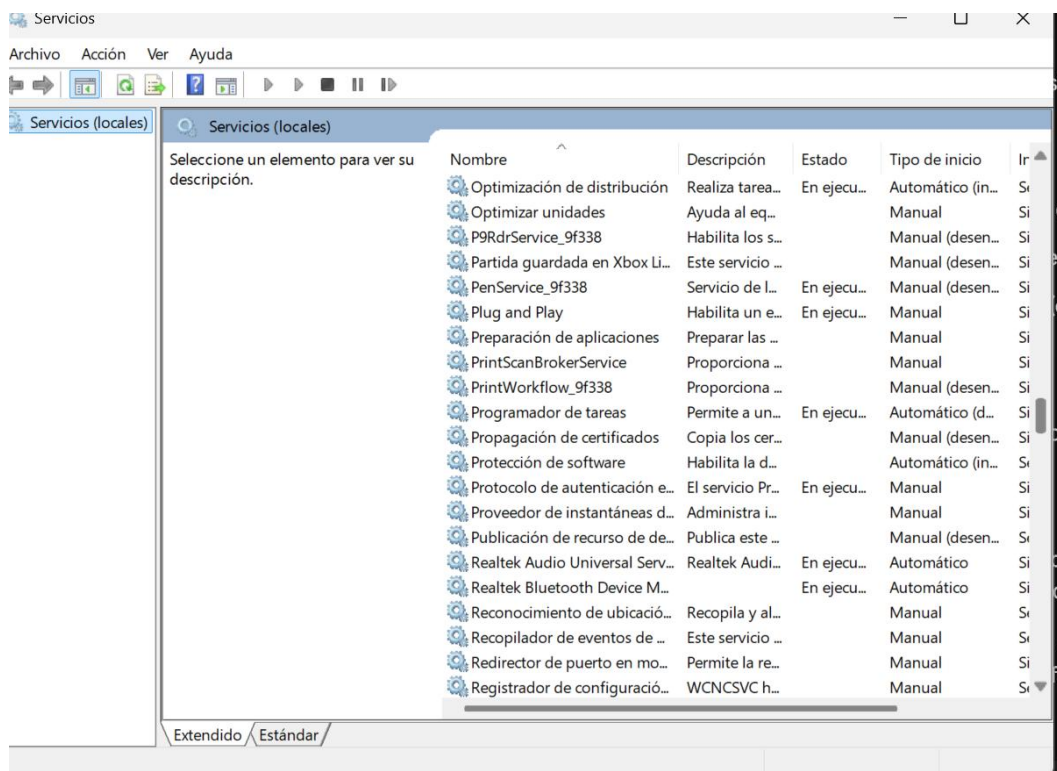
Capturas del recorrido real para dejar la base de datos conectada y funcionando.



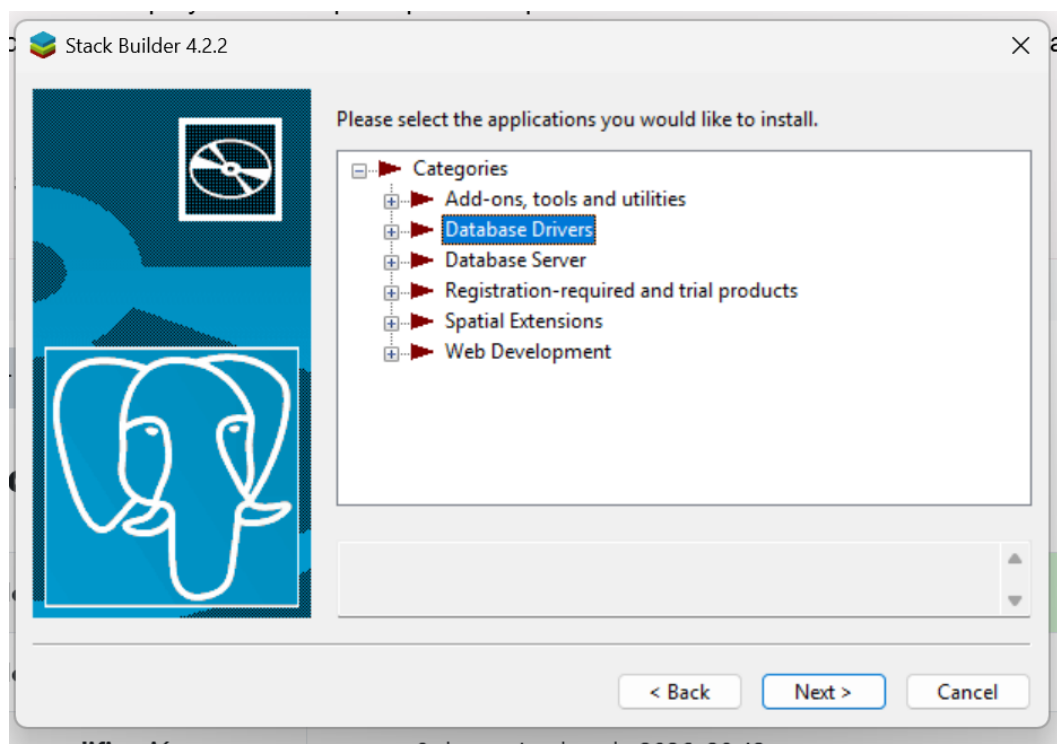
1. DBeaver mostraba la conexión como "Disconnected" — primer paso: reconectar.



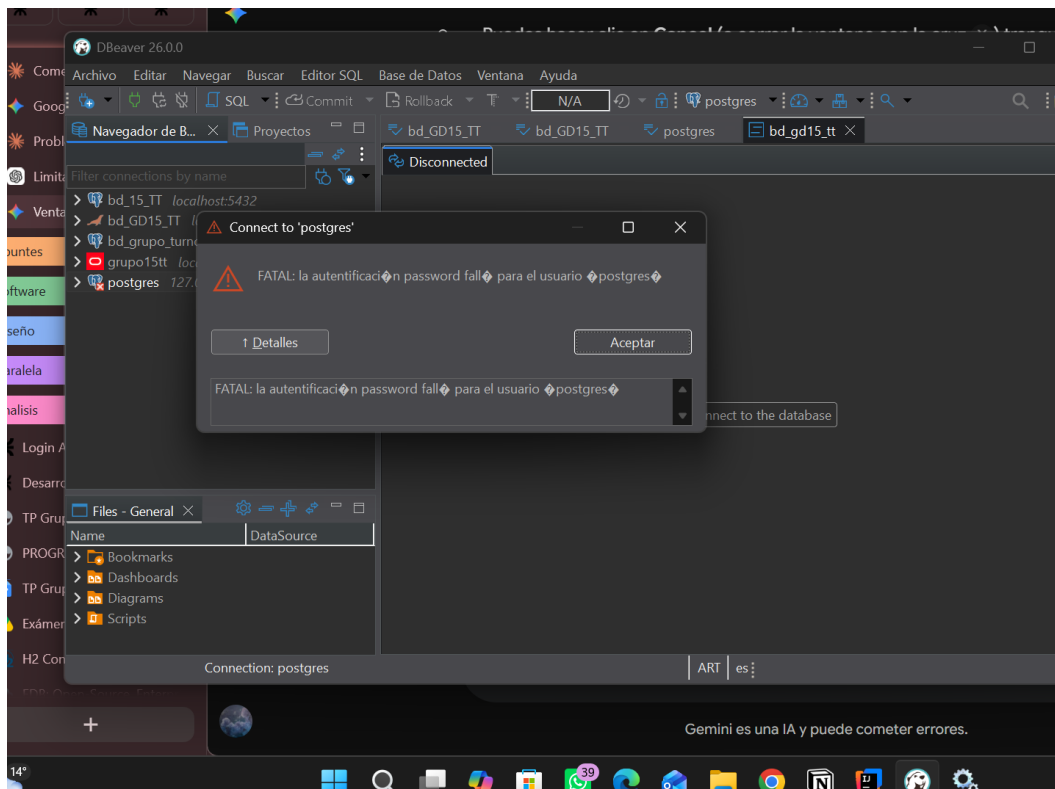
2. Primer intento fallido: "Connection refused" — el servicio de PostgreSQL no estaba iniciado.



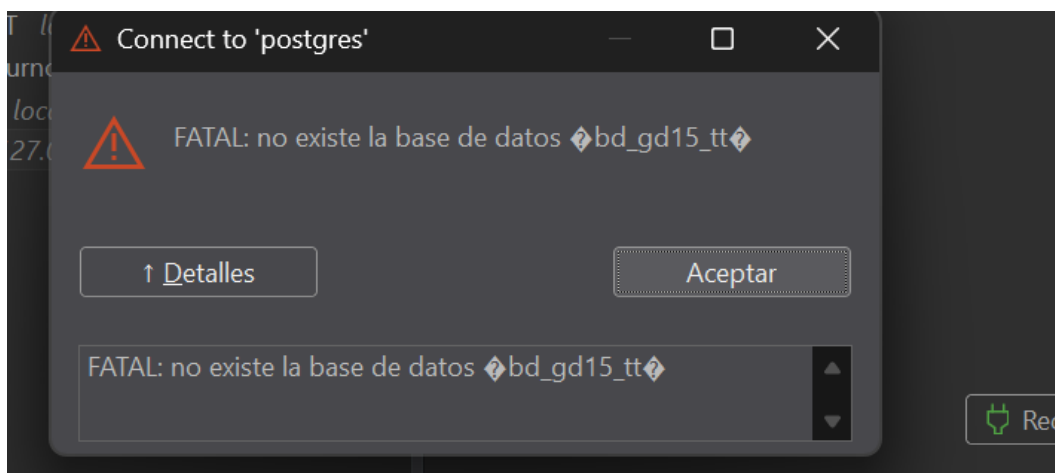
3. Se revisó en los Servicios de Windows que el servicio de PostgreSQL estuviera en ejecución.



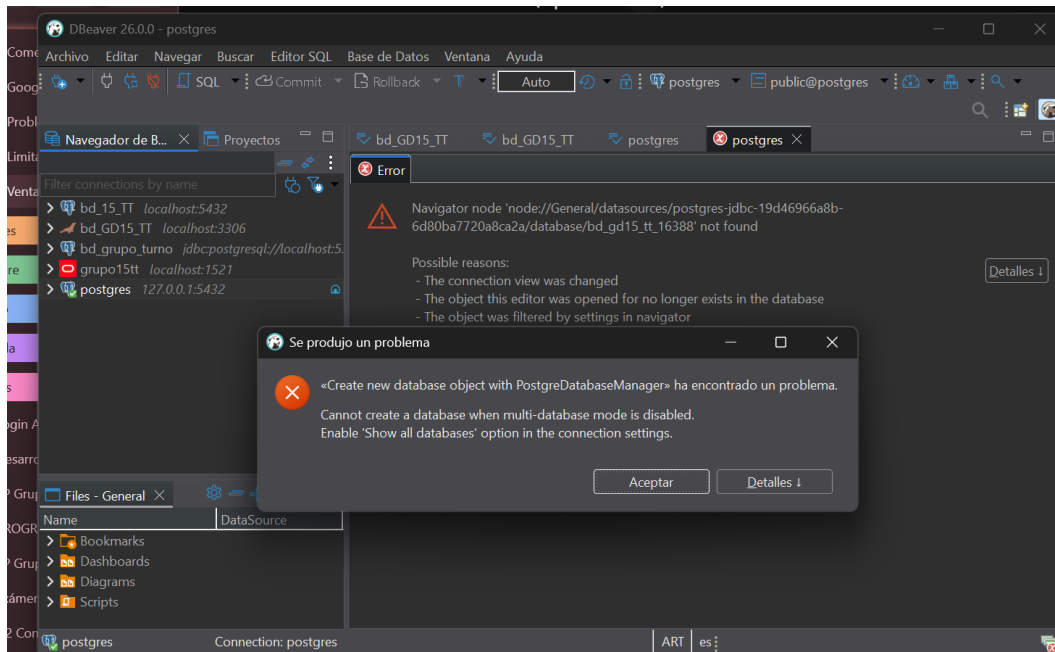
4. Instalación de los Database Drivers de PostgreSQL desde Stack Builder.



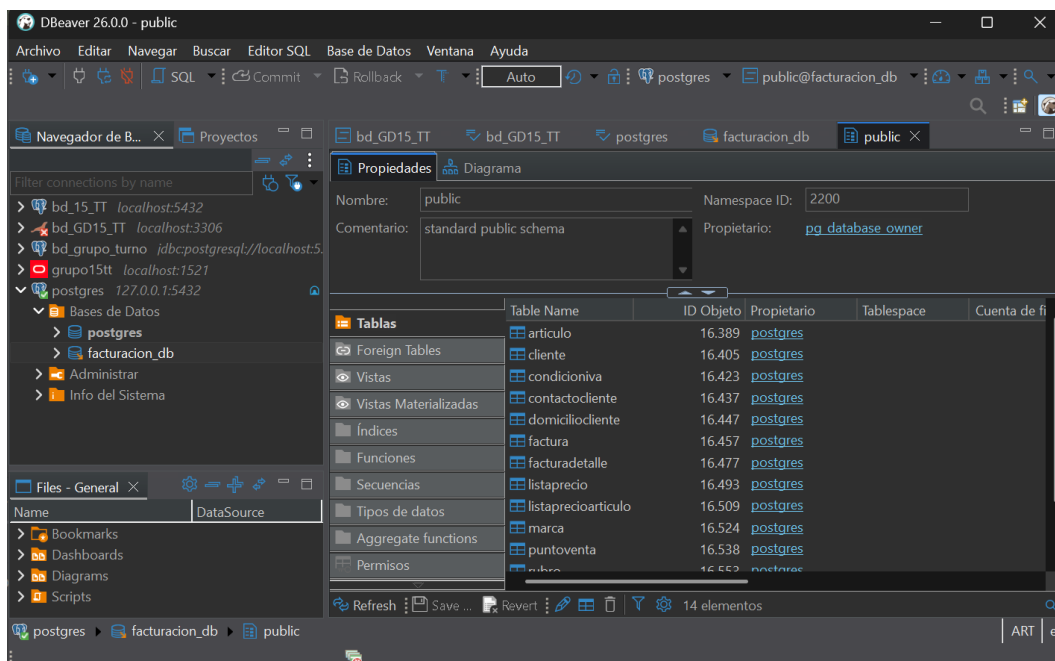
5. Nuevo error: "la autenticación password falló" — la contraseña del usuario postgres no coincidía.



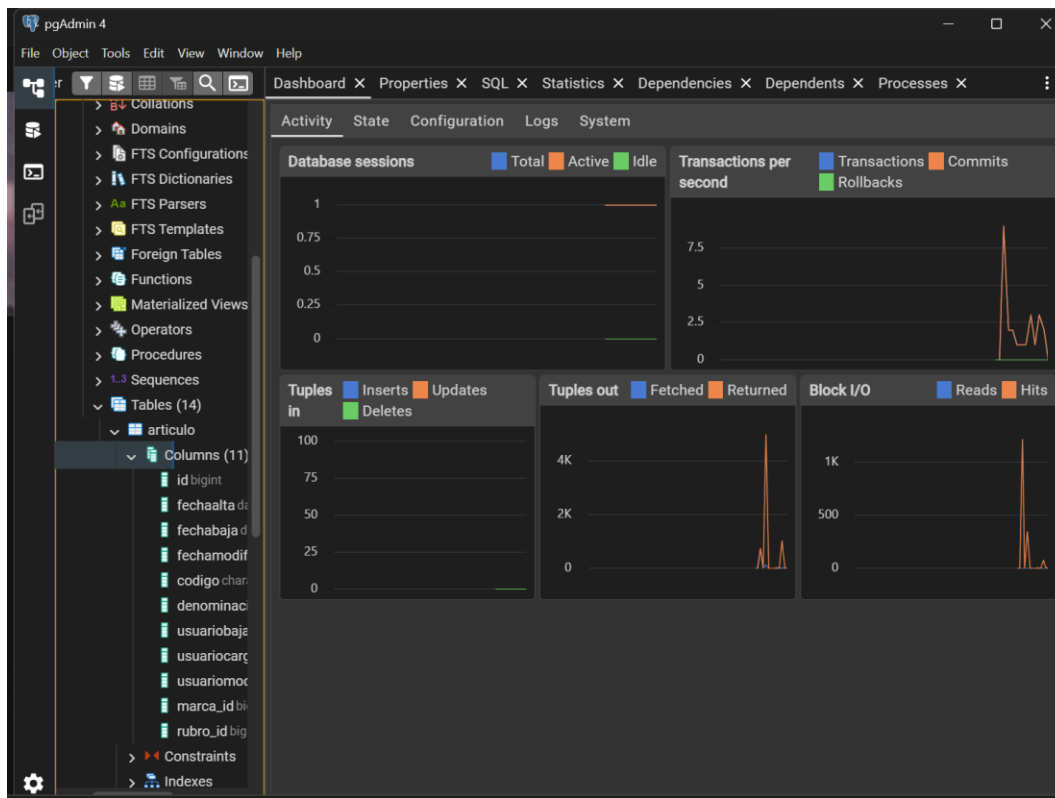
6. Error "no existe la base de datos" — la base todavía no había sido creada.



7. Para poder crearla desde DBeaver hubo que habilitar la opción "Show all databases" en la conexión.



8. Conexión exitosa: la base facturacion_db creada, con sus 14 tablas ya generadas por Hibernate.



9. Verificación final en pgAdmin 4: la tabla articulo con sus 11 columnas, reflejando exactamente el modelo Java (EntityId + AuditoriaApp + campos propios + relaciones).